

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»**

**GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)**

**Field of Study
09.03.01 – «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»**

**Тема /
Topic**

**Исследование Kubernetes как платформы для бизнес-логики с
использованием Custom Resource Definitions и паттерна Operator /
Exploring Kubernetes as a Platform for Business Logic Using Custom
Resource Definitions and the Operator Pattern**

**Работу выполнил /
Thesis is executed by**

**Бобкунов Евгений /
Bobkunov Evgeny**

подпись / signature

**Руководитель
выпускной
квалификационной
работы /
Supervisor of
Graduation Thesis**

Awwal Ishiaku

подпись / signature

**Консультанты /
Consultants**

Dmitrii Creed

подпись / signature

Иннополис, Innopolis, 2026

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	5
1.3	Research Questions	6
1.4	Research Objectives and Contributions	6
1.5	Research Methodology	8
1.6	Thesis Organization	9
1.7	Summary	10
2	Literature Review	11
2.1	Introduction	11
2.2	Kubernetes Architecture and Core Concepts	11
2.2.1	Container Orchestration and Kubernetes	11
2.2.2	Declarative API Model	12
2.2.3	Controller Pattern and Reconciliation Loop	13
2.3	Custom Resource Definitions	13
2.3.1	Extending the Kubernetes API	13
2.3.2	CRD Features and Capabilities	14
2.3.3	CRD Performance and Scalability	14

2.4	The Operator Pattern	15
2.4.1	Operator Fundamentals	15
2.4.2	Operator Development Frameworks	15
2.4.3	Operator Use Cases and Applications	16
2.5	Business Logic in Kubernetes	16
2.5.1	Modeling Business Entities as Custom Resources	16
2.5.2	Kubernetes as a Database	17
2.6	Related Systems and Approaches	17
2.6.1	Infrastructure as Code Platforms	17
2.6.2	Service Mesh and Infrastructure Patterns	18
2.6.3	Event-Driven Architectures	18
2.7	Observability in Cloud-Native Systems	19
2.7.1	The Three Pillars of Observability	19
2.7.2	Distributed Tracing	19
2.7.3	Observability for Operators	20
2.8	Performance and Scalability Studies	20
2.8.1	Kubernetes Performance Characteristics	20
2.8.2	Comparative Performance Studies	21
2.9	Challenges and Considerations	21
2.9.1	Complexity and Operational Overhead	21
2.9.2	Security Considerations	22
2.9.3	State Management and Data Persistence	22
2.10	Research Gaps	23
2.11	Summary	23

Chapter 1

Introduction

1.1 Background and Context

The rapid evolution of cloud-native computing has fundamentally transformed how modern software systems are architected, deployed, and operated. At the core of this transformation lies Kubernetes, which has emerged as the de facto standard for container orchestration since its release in 2014 [1]. Initially developed by Google and later donated to the Cloud Native Computing Foundation (CNCF), Kubernetes provides robust capabilities for automating deployment, scaling, and management of containerized applications [2].

However, Kubernetes offers far more than container orchestration capabilities. Its extensible architecture, built around a declarative API model and a control plane pattern, enables it to function as a universal control plane for managing arbitrary resources [3]. Through Custom Resource Definitions (CRDs) and the Operator pattern, Kubernetes can be extended to manage not only infrastructure components but potentially business logic itself [4], [5].

Modern cloud systems strive for maximum automation and flexibility. Orga-

nizations are increasingly seeking unified platforms that can manage both infrastructure and application logic through consistent interfaces. The declarative nature of Kubernetes, combined with its powerful extension mechanisms, positions it as a potential candidate for implementing business logic, not just orchestrating containers. This paradigm shift raises fundamental questions about the boundaries of the Infrastructure-as-Code approach and whether Kubernetes can serve as a unified control plane for both infrastructure and application concerns [6].

1.2 Problem Statement

Despite the widespread adoption of Kubernetes for infrastructure management, there remains a lack of systematic study regarding its applicability for implementing business logic using CRDs and the Operator pattern. Traditional application architectures typically separate business logic implementation from infrastructure concerns, with applications deployed as microservices that implement domain-specific functionality through conventional programming approaches.

Several projects have demonstrated the power of extending Kubernetes through CRDs and Operators, including Crossplane for infrastructure provisioning [7], Kubeflow for machine learning workflows, and various database operators. However, most existing solutions focus on infrastructure management or specialized workloads rather than general-purpose business logic [8].

The main problem lies in understanding whether and how Kubernetes, through CRDs and Operators, can effectively serve as a platform for implementing business logic. Key questions include: What are the performance characteristics of such an approach? How does it compare to traditional REST API-based architectures? What are the operational implications? What types of business

problems are well-suited to this paradigm?

Solving this problem is important for several reasons. First, it could enable organizations to consolidate their technology stacks by using Kubernetes as a unified platform. Second, it may provide new patterns for building highly automated, self-healing business applications. Third, it could help determine clear boundaries for when this approach is beneficial versus when traditional architectures should be preferred.

1.3 Research Questions

This thesis investigates the following research questions:

RQ1: How can business logic be effectively modeled and implemented using Kubernetes Custom Resource Definitions and the Operator pattern?

RQ2: What are the performance characteristics of a business logic implementation based on CRDs and Operators compared to traditional REST API architectures?

RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

RQ4: Under what circumstances is the CRD/Operator approach advantageous for implementing business logic, and what are its limitations?

1.4 Research Objectives and Contributions

The primary objective of this research is to explore and evaluate Kubernetes as a platform for implementing business logic through Custom Resource Definitions and the Operator pattern. Specific objectives include:

1. Study existing approaches to extending the Kubernetes API through CRDs and Operators, analyzing patterns and best practices established by the cloud-native community.
2. Design and develop a prototype application that implements representative business logic using CRDs and Operators, demonstrating the feasibility of this approach.
3. Establish comprehensive observability for the prototype system using cloud-native tools including Prometheus, Grafana, and OpenTelemetry to enable detailed performance analysis.
4. Conduct experimental comparisons between the CRD/Operator-based implementation and a traditional architecture using PostgreSQL with REST API.
5. Analyze results across multiple dimensions including reaction time to events, implementation complexity, observability capabilities, and system reliability.

The expected contributions of this work include:

- A systematic analysis of using Kubernetes CRDs and Operators for business logic implementation, filling a gap in current research.
- A working prototype demonstrating practical patterns for modeling business entities and processes as Kubernetes custom resources.
- Quantitative performance comparisons providing empirical evidence of the strengths and weaknesses of this approach.

- Practical recommendations and guidelines for practitioners considering this architectural approach, including identification of suitable use cases and potential pitfalls.
- Insights into the boundaries of Kubernetes extensibility and the Infrastructure-as-Code paradigm.

1.5 Research Methodology

This research employs a mixed-methods approach combining system design, implementation, and experimental evaluation:

Phase 1 - Literature Review and Architecture Design: Conduct comprehensive review of existing literature on Kubernetes extensibility, CRDs, Operators, and related patterns. Design the architecture for both the CRD/Operator-based system and the baseline traditional system.

Phase 2 - Implementation: Develop the prototype systems using appropriate tools and frameworks. The CRD/Operator implementation will utilize Kube-builder for scaffolding and controller development. Both systems will implement equivalent business logic to enable fair comparison.

Phase 3 - Observability Setup: Configure comprehensive monitoring using Prometheus for metrics collection, Grafana for visualization, and OpenTelemetry for distributed tracing. Implement GitOps practices using ArgoCD for deployment management.

Phase 4 - Experimental Evaluation: Conduct controlled experiments measuring key performance indicators including:

- Reaction time to events (how quickly the system responds to changes in

business entity state)

- Implementation complexity (lines of code, cognitive load for developers)
- Observability coverage (metrics, traces, and logs available)
- Reliability (number of incidents or errors under test conditions)

Phase 5 - Analysis and Conclusions: Analyze experimental results, synthesize findings, and formulate conclusions on the applicability of the approach.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 - Literature Review: Provides comprehensive background on Kubernetes architecture, Custom Resource Definitions, the Operator pattern, and related work in extending Kubernetes for business logic. Reviews relevant observability tools and GitOps practices.

Chapter 3 - System Design and Architecture: Describes the design of both the CRD/Operator-based system and the traditional baseline system. Details the business logic domain selected for implementation and architectural decisions.

Chapter 4 - Implementation: Covers implementation details including CRD specifications, Operator controller logic, observability instrumentation, and deployment configuration.

Chapter 5 - Experimental Evaluation: Presents the experimental setup, methodology, and results of performance comparisons between the two approaches.

Chapter 6 - Discussion: Analyzes results, discusses implications, identifies suitable use cases, and provides recommendations for practitioners.

Chapter 7 - Conclusion: Summarizes key findings, discusses limitations, and suggests directions for future work.

1.7 Summary

This chapter has introduced the research problem of using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern. While Kubernetes has proven highly successful for infrastructure management, its applicability for implementing business logic remains understudied. This thesis aims to address this gap through systematic design, implementation, and evaluation of both CRD/Operator-based and traditional architectures. The expected outcomes include practical insights into when and how this approach can be beneficial, contributing to the broader understanding of cloud-native application development patterns.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the literature and technologies relevant to exploring Kubernetes as a platform for business logic implementation. The chapter is organized into several sections covering Kubernetes architecture and extensibility, Custom Resource Definitions and Operators, related approaches to extending Kubernetes, observability in cloud-native systems, and comparative studies of container orchestration platforms.

2.2 Kubernetes Architecture and Core Concepts

2.2.1 Container Orchestration and Kubernetes

Kubernetes, initially developed by Google and open-sourced in 2014, has become the leading platform for container orchestration [1]. The system builds upon 15 years of experience running production workloads at Google, combined with

best-of-breed ideas from the open-source community [2]. Kubernetes automates deployment, scaling, and management of containerized applications, providing a platform-agnostic abstraction layer over underlying infrastructure [9].

The architecture of Kubernetes follows a control plane pattern, where a centralized control plane manages the desired state of the cluster, and worker nodes execute containerized workloads [10]. The control plane consists of several key components: the API server, which exposes the Kubernetes API and serves as the entry point for all operations; etcd, a distributed key-value store that maintains cluster state; the scheduler, which assigns pods to nodes; and the controller manager, which runs controllers to reconcile actual state with desired state [11].

2.2.2 Declarative API Model

A distinguishing feature of Kubernetes is its declarative API model [12]. Rather than imperatively instructing the system on how to perform operations, users declare the desired state of resources, and Kubernetes automatically reconciles the actual state to match. This declarative approach enables powerful automation capabilities and forms the foundation for extending Kubernetes functionality.

The Kubernetes API is organized into API groups and versions, providing a structured approach to API evolution and extensibility [13]. Core resources like Pods, Services, and Deployments are part of the built-in API, while custom resources can be added through extension mechanisms.

2.2.3 Controller Pattern and Reconciliation Loop

Kubernetes extensively uses the controller pattern for managing resources [14]. A controller watches for changes to resources of interest and takes actions to reconcile the actual state with the desired state specified in the resource definition. This reconciliation loop runs continuously, enabling self-healing and automated management capabilities.

Controllers operate independently and asynchronously, communicating only through the Kubernetes API server. This architectural pattern enables high scalability and resilience, as controllers can be distributed across multiple instances and automatically recover from failures.

2.3 Custom Resource Definitions

2.3.1 Extending the Kubernetes API

Kubernetes provides two primary mechanisms for extending its API: Custom Resource Definitions (CRDs) and API Aggregation [15]. CRDs represent a declarative approach to defining new resource types without requiring custom API server code, while API aggregation allows for more specialized implementations with custom API servers.

CRDs were introduced in Kubernetes 1.7 as a replacement for the earlier Third-Party Resources (TPRs), addressing limitations in validation, versioning, and performance [16]. A CRD defines a new resource type, including its API group, version, kind, and schema, making it a first-class citizen in the Kubernetes API [4].

2.3.2 CRD Features and Capabilities

Modern CRDs support advanced features including schema validation using OpenAPI v3, multiple versions with conversion webhooks, status subresources for privilege separation, custom columns in kubectl output, and structural schemas for efficient storage and processing [17]. These capabilities enable CRDs to provide rich, production-ready APIs comparable to built-in Kubernetes resources.

Research by Mia Platform demonstrates how CRDs enable developers to model custom workflows, advanced configurations, and support for non-standard infrastructure components that go beyond Kubernetes' built-in resources [16]. The flexibility of CRDs has made them a cornerstone of cloud-native extensibility, with hundreds of widely-used CRDs available in the ecosystem.

2.3.3 CRD Performance and Scalability

The performance characteristics of CRDs depend heavily on the underlying etcd datastore, which stores all cluster state [18]. Research on etcd performance optimization shows that at scale, write latency can become a bottleneck, particularly for clusters with many CRD instances [19]. Studies indicate that once etcd capacity exceeds approximately 40GB, operation latency increases significantly, potentially impacting CRD-based applications [20].

To address scalability concerns, recent work has proposed techniques including consistent reads from cache, which can reduce API server CPU usage by 30% and etcd CPU usage by 25% in large clusters [21]. Organizations deploying CRD-intensive applications must carefully consider etcd capacity planning and implement appropriate monitoring.

2.4 The Operator Pattern

2.4.1 Operator Fundamentals

The Operator pattern represents a methodology for packaging, deploying, and managing Kubernetes applications by extending the Kubernetes API [5]. Operators combine CRDs with custom controllers to encode operational knowledge about managing complex applications, effectively acting as automated site reliability engineers [22].

Operators capture the key aim of human operators who manage services—they have deep knowledge of how systems should behave, how to deploy them, and how to react to problems [23]. By codifying this operational knowledge, Operators enable applications to be self-managing, automatically handling tasks like upgrades, backups, scaling, and failure recovery.

2.4.2 Operator Development Frameworks

Several frameworks facilitate Operator development, with Kubebuilder and Operator SDK being the most prominent [8]. Kubebuilder, built on top of controller-runtime and controller-tools libraries, provides scaffolding for creating CRDs and controllers using Go [24]. The framework automates much of the boilerplate code generation, allowing developers to focus on implementing business logic in the reconciliation loop.

Research on Operator development best practices emphasizes the importance of proper RBAC configuration, efficient handling of custom resources, and robust error handling [25]. Studies analyzing Operator bugs across 36 Kubernetes operators identified common failure patterns related to state management, error

handling, and race conditions [26].

2.4.3 Operator Use Cases and Applications

Operators have been successfully applied to numerous domains including database management (PostgreSQL, Cassandra), monitoring systems (Prometheus, Grafana), service meshes (Istio), and machine learning platforms (Kubeflow) [25]. The PRISTINE project demonstrates using a Kubernetes CRD-based operator for priority-aware resource orchestration in cloud-native environments [27].

For infrastructure provisioning, Crossplane exemplifies using Operators to manage cloud resources declaratively through Kubernetes [7]. This approach enables treating infrastructure as Kubernetes resources, managed through the same GitOps workflows as application deployments.

2.5 Business Logic in Kubernetes

2.5.1 Modeling Business Entities as Custom Resources

The concept of using Kubernetes for business logic implementation remains relatively unexplored in academic literature. A notable discussion by IT Next explores getting down to business logic with Kubernetes Operators, suggesting that Operators allow defining and encapsulating custom units of business logic within the Kubernetes ecosystem [28].

The declarative nature of Kubernetes resources provides a natural fit for representing business entities and their desired states. Business processes can be modeled as state machines, with custom controllers implementing the business logic that transitions entities between states in response to events or conditions.

2.5.2 Kubernetes as a Database

Some perspectives suggest viewing Kubernetes itself as a database, where the declarative API and persistent storage in etcd enable treating Kubernetes as a control plane for arbitrary data [29]. This paradigm shift views Kubernetes not merely as a container orchestrator but as a programmable platform with built-in CRUD operations, watches, and schema validation.

Research on using Kubernetes as a cloud services provisioner demonstrates how CRDs can represent external services and resources, managed declaratively through Kubernetes APIs [30]. This approach provides a unified interface for managing heterogeneous resources across multiple providers.

2.6 Related Systems and Approaches

2.6.1 Infrastructure as Code Platforms

Traditional Infrastructure as Code (IaC) tools like Terraform, Ansible, and CloudFormation provide declarative approaches to infrastructure provisioning [6]. While similar in declaring desired state, these tools typically use imperative execution models and lack the continuous reconciliation capabilities inherent to Kubernetes controllers [31].

The GitOps methodology, popularized by tools like ArgoCD and Flux, extends IaC principles by using Git as the single source of truth and implementing continuous delivery through pull-based reconciliation [32], [33]. GitOps shares philosophical alignment with Kubernetes' declarative model, making it a natural fit for managing Kubernetes resources including CRDs.

2.6.2 Service Mesh and Infrastructure Patterns

Service mesh technologies like Istio and Linkerd demonstrate advanced usage of Kubernetes extensibility for managing service-to-service communication [34], [35]. These systems use CRDs extensively to define traffic management, security policies, and observability configurations, providing examples of complex operational logic implemented through Kubernetes extension mechanisms [36].

Research comparing service mesh architectures reveals trade-offs between feature richness and operational complexity, with implications for designing custom business logic controllers [37]. The sidecar pattern commonly used in service meshes offers lessons for separating concerns in business logic implementations.

2.6.3 Event-Driven Architectures

Event-driven architectures provide an alternative approach to implementing business logic, where components react to events rather than following procedural execution flows [38]. Kubernetes naturally supports event-driven patterns through its watch mechanism, enabling controllers to react to resource changes in real-time.

The integration of event-driven concepts with Kubernetes has been explored in various contexts, including IoT orchestration and microservices communication [39], [40]. These patterns are relevant to understanding how business logic can be triggered and coordinated through Kubernetes events.

2.7 Observability in Cloud-Native Systems

2.7.1 The Three Pillars of Observability

Observability in distributed systems rests on three pillars: metrics, logs, and traces [41]. Metrics provide quantitative measurements of system behavior over time. Logs capture discrete events and messages. Traces track requests as they propagate through distributed systems. Together, these pillars enable understanding system behavior and diagnosing issues.

Prometheus has emerged as the de facto standard for metrics collection in Kubernetes environments [42]. Its pull-based model, powerful query language (PromQL), and native Kubernetes service discovery make it well-suited for cloud-native monitoring. Grafana typically serves as the visualization layer, providing dashboards built on Prometheus data sources [43].

2.7.2 Distributed Tracing

OpenTelemetry represents the industry standard for distributed tracing and observability instrumentation [44]. Research on tracing and metrics design patterns for cloud-native applications demonstrates best practices for instrumenting Kubernetes-based systems [45]. For CRD-based applications, proper instrumentation of controllers and business logic is essential for understanding system behavior.

Distributed tracing becomes particularly important when business logic spans multiple reconciliation loops and controllers, as it enables tracking the cascading changes of objects through the Kubernetes control plane [46].

2.7.3 Observability for Operators

Specific considerations apply when implementing observability for Kubernetes Operators. Controllers should expose metrics about reconciliation loop performance, error rates, and business-specific indicators. Research on Operator reliability emphasizes the importance of comprehensive observability for maintaining production systems [47].

The integration of Prometheus with Kubebuilder-based Operators is well-established, with frameworks providing automatic instrumentation of common controller metrics [48]. However, domain-specific business metrics require custom implementation within the reconciliation logic.

2.8 Performance and Scalability Studies

2.8.1 Kubernetes Performance Characteristics

Multiple studies have examined Kubernetes performance under various conditions. Research on etcd deployment impact demonstrates that control plane performance and application performance both depend critically on etcd performance characteristics [49]. Write latency and throughput limitations in etcd can become bottlenecks for CRD-intensive workloads.

Studies of Kubernetes at scale reveal challenges in maintaining performance as cluster size grows [50]. For edge computing scenarios, research proposes eventually consistent approaches to improve availability and performance compared to etcd's strong consistency model [20].

2.8.2 Comparative Performance Studies

Comparative analyses of container orchestration platforms provide context for understanding Kubernetes performance characteristics. Research comparing Kubernetes with serverless computing architectures reveals trade-offs between control, flexibility, and operational overhead [51]. Serverless platforms excel in scenarios with variable demand and event-driven workloads, while Kubernetes provides greater control for stateful applications.

Studies examining Kubernetes against traditional REST API implementations are limited. This research gap motivates the experimental comparison component of this thesis, providing empirical evidence for the performance implications of using Kubernetes for business logic.

2.9 Challenges and Considerations

2.9.1 Complexity and Operational Overhead

While Kubernetes provides powerful capabilities, it introduces significant complexity and operational overhead [1]. Managing a production Kubernetes cluster requires expertise in networking, security, storage, and distributed systems. Organizations must weigh these operational costs against the benefits of Kubernetes' automation and standardization.

For business logic implementations specifically, additional complexity arises from the need to design appropriate CRD schemas, implement robust controllers, and handle edge cases in the reconciliation logic. The learning curve for Kubernetes development is substantial, potentially impacting development velocity.

2.9.2 Security Considerations

Implementing business logic in Kubernetes raises important security considerations. CRDs and Operators typically require elevated privileges to manage cluster resources, necessitating careful RBAC design [52]. Research on Kubernetes secret security emphasizes the importance of properly securing sensitive data used by controllers [52].

Validation and admission webhooks can help enforce security policies and data integrity constraints for custom resources [53]. However, these mechanisms add complexity and potential performance overhead that must be considered in system design.

2.9.3 State Management and Data Persistence

Using Kubernetes for business logic raises questions about state management and data persistence. While etcd provides reliable storage for resource definitions, it may not be suitable for large volumes of transactional business data. Hybrid approaches that use CRDs for state machine representation while storing bulk data in external databases may be necessary.

Research on Kubernetes storage orchestration and persistence patterns provides guidance for designing stateful applications [54]. The choice of storage backend and data modeling approach significantly impacts system performance and reliability.

2.10 Research Gaps

Despite extensive research on Kubernetes architecture, CRDs, and Operators, several gaps exist in current literature:

1. Limited systematic study of using Kubernetes for general-purpose business logic implementation, with most examples focused on infrastructure or specialized domains.
2. Insufficient quantitative performance comparisons between CRD/Operator-based approaches and traditional architectures for equivalent business logic.
3. Lack of comprehensive guidelines on when the CRD/Operator pattern is appropriate for business applications versus when traditional architectures should be preferred.
4. Limited research on implementation complexity and developer experience when using Kubernetes as a business logic platform.
5. Insufficient exploration of hybrid approaches that combine CRD-based orchestration with traditional application architectures.

This thesis aims to address these gaps through systematic design, implementation, and evaluation of Kubernetes-based business logic using CRDs and Operators.

2.11 Summary

This chapter has reviewed the relevant literature on Kubernetes architecture, extensibility mechanisms, and related technologies. Kubernetes provides a pow-

erful platform for building cloud-native applications through its declarative API, CRDs, and Operator pattern. While extensively used for infrastructure management, its application to general business logic remains understudied. The next chapter describes the design of systems to explore this research question.

Bibliography cited

- [1] Kubernetes, *Production-grade container orchestration*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io>
- [2] Cloud Native Computing Foundation, *Cloud native computing foundation*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.cncf.io>
- [3] Kubernetes, *Extending kubernetes*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/>
- [4] Kubernetes, *Custom resources*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
- [5] Kubernetes, *Operator pattern*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- [6] Microtica, *Infrastructure as code in devops: Benefits, tools, and best practices*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://www.microtica.com/blog/what-is-infrastructure-as-code>
- [7] Crossplane, *Crossplane is the cloud-native framework for platform engineering*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.crossplane.io>

- [8] Kubernetes SIG, *Kubebuilder: Sdk for building kubernetes apis using crds*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://github.com/kubernetes-sigs/kubebuilder>
- [9] Datadog, *What is container orchestration? how it works & use cases*, Accessed: 2025-10-31, 2023. [Online]. Available: <https://www.datadoghq.com/knowledge-center/container-orchestration/>
- [10] Kubernetes, *Cluster architecture*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>
- [11] Red Hat OpenShift, *Control plane architecture*, Accessed: 2025-10-31, 2023. [Online]. Available: https://docs.redhat.com/en/documentation/openshift_dedicated/4/html/architecture/control-plane
- [12] A. Cloud, *Fundamentals of declarative application management in kubernetes*, Accessed: 2025-10-31, 2023. [Online]. Available: <https://www.alibabacloud.com/blog/596265>
- [13] Kubernetes, *The kubernetes api*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>
- [14] Kubernetes, *Controllers*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>
- [15] Kubernetes, *Extending the kubernetes api*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/>

- [16] M. Platform, *Custom resource definition in kubernetes*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://mia-platform.eu/blog/custom-resource-definition-in-kubernetes/>
- [17] Kubernetes, *Extend the kubernetes api with custom resource definitions*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>
- [18] Plural, *EtcD in kubernetes: The essential guide*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://www.plural.sh/blog/etcd-kubernetes-guide/>
- [19] CNCF, “Performance optimization of etcd in web scale data scenario,” *CNCF Blog*, May 2019. [Online]. Available: <https://www.cncf.io/blog/2019/05/09/performance-optimization-of-etcd-in-web-scale-data-scenario/>
- [20] A. Jeffery, H. Howard, and R. Mortier, “Rearchitecting kubernetes for the edge,” *ACM Digital Library*, Apr. 2021. DOI: [10.1145/3434770.3459730](https://doi.org/10.1145/3434770.3459730) [Online]. Available: <https://doi.org/10.1145/3434770.3459730>
- [21] Kubernetes, *Accelerating cluster performance with consistent reads from cache*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/blog/2024/08/15/consistent-read-from-cache-beta/>
- [22] Red Hat, *Operator framework*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://operatorframework.io/>

- [23] CNCF, *Kubernetes operators: What are they? some examples*, Accessed: 2025-10-31, 2022. [Online]. Available: <https://www.cncf.io/blog/2022/06/15/kubernetes-operators-what-are-they-some-examples/>
- [24] Kubebuilder, *The kubebuilder book*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://book.kubebuilder.io/>
- [25] Trilio, *Kubernetes operators explained: Key concepts and practical applications*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://trilio.io/kubernetes-management-tools/kubernetes-operators/>
- [26] Q. Xu, Y. Gao, and J. Wei, “An empirical study on kubernetes operator bugs,” *ACM Digital Library*, 2024, Accessed: 2025-10-31. DOI: [10.1145/3650212.3680396](https://doi.org/10.1145/3650212.3680396) [Online]. Available: <https://doi.org/10.1145/3650212.3680396>
- [27] P. Josyula, A. Kumar, and G. Hiremath, “Pristine: Priority-aware smart resource orchestration engine for cloud-native applications,” 2025. DOI: [10.1109/Cloud-Summit64795.2025.00036](https://doi.org/10.1109/Cloud-Summit64795.2025.00036)
- [28] I. Next, *Kubernetes operators... getting down to business (logic)*, Accessed: 2025-10-31, 2023. [Online]. Available: <https://itnext.io/kubernetes-operators-getting-down-to-business-logic-93d6f5303813>
- [29] Nilebits, *Kubernetes as a database? what you need to know about crds*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.nilebits.com/blog/2024/06/kubernetes-as-a-database-need-know-crds/>
- [30] CNCF, *Kubernetes as a cloud services provisioner*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.cncf.io/blog/2021/02/23/>

- kubernetes-as-a-cloud-services-provisioner-what-are-custom-resource-definitions/
- [31] Spacelift, *Gitops vs infrastructure as code (iac): Differences & use cases*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://spacelift.io/blog/gitops-vs-iac>
 - [32] GitLab, *What is gitops?* Accessed: 2025-10-31, 2023. [Online]. Available: <https://about.gitlab.com/topics/gitops/>
 - [33] A. Project, *Argo cd - declarative gitops cd for kubernetes*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://argo-cd.readthedocs.io>
 - [34] P. Josyula and A. Kumar, “Architectural trade-off analysis of modern service mesh designs: A qualitative atam study,” 2025. DOI: [10.1109/CICTN64563.2025.10932541](https://doi.org/10.1109/CICTN64563.2025.10932541)
 - [35] Solo.io, *Linkerd vs. istio: 7 key differences*, Accessed: 2025-10-31, 2023. [Online]. Available: <https://www.solo.io/topics/istio/linkerd-vs-istio>
 - [36] R. R. Karn, R. Das, D. R. Pant, J. Heikkonen, and R. Kanth, “Automated testing and resilience of microservice’s network-link using istio service mesh,” 2022. DOI: [10.23919/FRUCT54823.2022.9770890](https://doi.org/10.23919/FRUCT54823.2022.9770890)
 - [37] A. B. Barr, O. Lavi, Y. Naor, S. Rampal, and J. Tavori, *Technical report: Performance comparison of service mesh frameworks: The mtls test case*, 2024. arXiv: [2411.02267](https://arxiv.org/abs/2411.02267) [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2411.02267>
 - [38] AWS, *Event-driven architecture*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://aws.amazon.com/event-driven-architecture/>

- [39] L. D'Agati, G. Tricomi, M. Arena, F. Longo, A. Puliafito, and G. Merlino, *Iot orchestration in the compute continuum: Integrating kubernetes with stack4things*, Accessed: 2025-10-31, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:279802457>
- [40] Kong, *What is microservices orchestration? tools and benefits*, Accessed: 2025-10-31, 2022. [Online]. Available: <https://konghq.com/blog/learning-center/microservices-orchestration>
- [41] Grafana, *Application observability based on opentelemetry and prometheus*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://grafana.com/products/cloud/application-observability/>
- [42] Prometheus, *Prometheus - monitoring system & time series database*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://prometheus.io>
- [43] G. Labs, *Prometheus with grafana: 5 compelling use cases*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.tigera.io/learn/guides/prometheus-monitoring/prometheus-grafana/>
- [44] OpenTelemetry, *Opentelemetry*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://opentelemetry.io>
- [45] C. Albuquerque and F. F. Correia, "Tracing and metrics design patterns for monitoring cloud-native applications," 2025. arXiv: 2510.02991 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2510.02991>
- [46] T. Ehira, D. Kotani, and Y. Okabe, "Distributed tracing for cascading changes of objects in the kubernetes control plane," 2024. arXiv: 2411.01336 [eess.SY]. [Online]. Available: <https://arxiv.org/abs/2411.01336>

- [47] Gremlin, *The ultimate guide to kubernetes high availability*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://www.gremlin.com/ultimate-guide-kubernetes-high-availability>
- [48] Kubebuilder, *Running and deploying the controller*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://book.kubebuilder.io/cronjob-tutorial/running>
- [49] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd deployment on kubernetes, istio, and application performance,” 2020. arXiv: 2004.00372 [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2004.00372>
- [50] S. Sagkriotis and D. Pezaros, *Scalable data plane caching for kubernetes*, 2022. DOI: 10.23919/CNSM55787.2022.9964497
- [51] G. Kambala, “Cloud-native architectures: A comparative analysis of kubernetes and serverless computing,” *Journal of Emerging Technologies and Innovative Research*, 2023. [Online]. Available: <https://www.jetir.org/papers/JETIR2304F29.pdf>
- [52] K. Gunathilake and I. Ekanayake, “K8s pro sentinel: Extend secret security in kubernetes cluster,” 2024. DOI: 10.1109/icitr64794.2024.10857769 [Online]. Available: <http://dx.doi.org/10.1109/ICITR64794.2024.10857769>
- [53] Kubernetes, *Dynamic admission control*, Accessed: 2025-10-31, 2024. [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>

- [54] Kubernetes, *Storage*, Accessed: 2025-10-31, 2024. [Online]. Available:
<https://kubernetes.io/docs/concepts/storage/>